

APPENDIX

A. Exploit Skeleton

This section shows what an exploit skeleton looks like.

Listing 7 A generic exploit skeleton showing the minimal structure expected in a generated PoC

```

async function exploit() {
  // Your proof-of-vulnerability code goes
  ↪ here
}
await exploit();

```

Listing 8 An example exploit skeleton generated by POCGEN, showing what a package-specific PoC should look like

```

async function exploit() {
  const lexer = require("dirty-json/lexer");
  const result = await lexer.lexString(...);
}
await exploit();

```

B. Prompts

This section documents the prompts used throughout our research.

Listing 9 Task-specific vulnerability-context scoring prompt.

Task: Scoring the usefulness of
 ↪ vulnerability-related context in the
 ↪ `user\_prompt`

Input context contains:

- user_prompt
 - response

Analysis steps:

1. Evaluate the prompt using the following
 - ↪ eight task-specific vulnerability-context
 - ↪ criteria:
 - candidate vulnerable API
 - vulnerability description
 - usage snippets
 - exploit skeleton
 - similar exploits
 - taint-path snippets
 - debugger output
 - code coverage
2. For each required criterion, inspect the
 - ↪ `user\_prompt` and `response` and assign a
 - ↪ numeric score from 0 to 1 using this
 - ↪ rubric:
 - 0: the context is absent, irrelevant, or
 - ↪ not useful for diagnosing the failure
 - ↪ and revising the prompt.
 - Between 0 and 1: the context provides
 - ↪ partially useful or actionable
 - ↪ guidance.
 - 1: the context provides highly useful
 - ↪ and actionable guidance for diagnosing
 - ↪ the failure and revising the prompt.
3. If a context is missing from the
 - ↪ `user\_prompt` or not present in the
 - ↪ `response`, include it in the output with
 - ↪ `score` = 0 and `explanation` = "missing
 - ↪ from input; treated as 0".
4. Prefer lower scores for generic, redundant,
 - ↪ or removable details and higher scores for
 - ↪ vulnerability-specific details (payloads,
 - ↪ API constraints, environment assumptions,
 - ↪ taint paths, concrete usage snippets).

Output guidance:

- Return ONLY valid JSON (no markdown or extra
 - ↪ text).
 - The top-level JSON value must be a list of
 - ↪ objects covering exactly the eight
 - ↪ required criteria.
 - Each object must have exactly these fields:
 - `criterion`: one of the eight required
 - ↪ criteria.
 - `score`: a number from 0 to 1.
 - `explanation`: a concise justification
 - ↪ for the score grounded in the
 - ↪ `user\_prompt` and the observed
 - ↪ `response`.
 - If any required context is missing, include
 - ↪ an object for it with `score` = 0 and
 - ↪ `explanation` = "missing from input;
 - ↪ treated as 0".
 - The JSON list order does not matter, but all
 - ↪ eight contexts must appear exactly once.
 - Do not output markdown, code fences,
 - ↪ headings, or extra commentary.
-

Listing 10 Feedback Prompt

Task: Analyze why PoC exploit generation
 ↳ failed and suggest improvements

Input context contains:

- user_prompt, response,
- ↳ proof_of_concept_code, execution_outputs
- vulnerability_fixing_commit, package,
- ↳ version, vulnerability_type, goals
- scores: per-criterion usefulness scores
- ↳ produced from the user_prompt and response

Analysis steps:

1. Examine the available vulnerability context
 - ↳ to understand the vulnerability and its
 - ↳ triggering conditions.
2. Compare proof_of_concept_code against
 - ↳ available evidence and verify API usage,
 - ↳ payload structure, and execution behavior.
3. Review execution_outputs for
 - ↳ syntax/runtime/assertion failures.
4. Use scores to identify which user_prompt
 - ↳ contexts were most and least helpful.
5. Identify why the user_prompt and generated
 - ↳ code failed to meet the goals.
6. Generate concise, actionable feedback to
 - ↳ improve the prompt.

Output contract (strict):

- Final output must contain exactly these two
 - ↳ numbered sections and nothing else:
 1. Why the PoC code failed?
 - 1.1. Based on ..., hence ...
 - 1.2. Based on ..., hence ...
 2. How to improve the user_prompt to
 - ↳ address this problem?
 - 2.1. Add/Remove/Replace ... to the
 - ↳ user_prompt would solve the 1.1
 - ↳ issue
 - 2.2. Add/Remove/Replace ... to the
 - ↳ user_prompt would solve the 1.2
 - ↳ issue
 - Keep the same numbering, question text, and
 - ↳ bullet style.
 - Limit to only 2 answers each questions.
 - Do not output code samples, JSON, markdown
 - ↳ wrappers, role labels, extra headings, or
 - ↳ extra commentary.
-

Listing 11 Reflective Evolution Prompt

Task: Prompt Evolution for Vulnerability

↳ Reproduction

Primary goal:

- Generate ONE improved userPrompt for PoC
- ↳ exploit generation (defensive purpose).

Input context contains:

- seed_prompt: prior prompt+feedback pair, if
 - ↳ available.
- example_prompts: list of prompt+feedback
 - ↳ examples.

Analysis steps:

1. Read all examples and feedback.
2. Extract concrete failure patterns from
 - ↳ feedback (missing code block, unclear
 - ↳ payload, wrong API usage, verbosity,
 - ↳ missing vulnerability-specific details).
3. Synthesize a new user prompt that directly
 - ↳ addresses these failures and preserves the
 - ↳ exploit-generation objective.
4. Reuse vulnerability details from examples
 - ↳ where useful.

Output contract (strict):

- Return ONLY the new user prompt text.
 - Do NOT output systemPrompt, JSON, markdown
 - ↳ wrappers, role labels, analysis sections,
 - ↳ headings, or extra commentary.
 - Do NOT claim missing examples unless
 - ↳ example_prompts is actually empty.
-

Listing 12 Exploit-generation prompt used by POCEVOLVE (LLM) in our proposed setting.

You are helping a security researcher write a
 ↳ proof-of-concept (PoC) exploit script for
 ↳ the npm package `{package}` in a
 ↳ controlled research environment. The
 ↳ purpose is to demonstrate that the
 ↳ {vulnerability_type} vulnerability exists
 ↳ so that developers can verify and patch it
 ↳ | not for any malicious use.

You can complete the following code snippet:

```
```js
async function exploit() {{
 // Your proof-of-concept code goes here
}}
await exploit();
```
```

Task:

Write a PoC script that demonstrates the
 ↳ {vulnerability_type} in `{package}` by
 ↳ triggering: {goal}.

Respond with the full PoV code and explain why
 ↳ it demonstrates the vulnerability.

If an exception is thrown, do not handle it |
 ↳ let it propagate so the failure is
 ↳ visible.

Enclose the PoC code in backticks and define
 ↳ it within a function named `exploit`.

Vulnerability Description
 {vulnerability_description}

Vulnerable API
 {vulnerable_api}

Vulnerability Fixing Commit
 Commit message: {commit_message}
 Code Changes:
 {code_changes}

Listing 13 Analyzer Prompt

You are a cybersecurity assistant. You will be given exactly four inputs:

1. Vulnerable package name
2. Vulnerable version
3. Commit Message
4. Commit Code Changes

Task: Using the package name, vulnerable version, commit message, and code changes, generate a

- concise vulnerability description that identifies the most likely vulnerability type and
- impacted code element.

Output requirements (strict):

- Return ONLY a single JSON object (no surrounding text, markdown, or commentary).
- The JSON MUST contain exactly these fields:
 - 'potential_vulnerability_type': one of the 5 (code-injection, command-injection, path-traversal, prototype-pollution, redos)
 - 'potential_vulnerable_API': a short identifier for the affected API (e.g., function name like parseUserInput, or file/path to import)
 - 'vulnerability_description': a vulnerability description follow provided examples

Examples of vulnerability descriptions (by type):

- code-injection: "The template() function in access-policy does not properly sanitize user input provided to the eval() function, resulting in arbitrary code execution."
- command-injection: "The main() function in corenlp-js-interface is vulnerable to command injection via unsanitized input, allowing attackers to execute arbitrary shell commands."
- path-traversal: "The asset-cache server is vulnerable to directory traversal attacks. Requesting paths like ../../%2f..%2fetc/passwd bypasses path restrictions and leaks sensitive files."
- prototype-pollution: "The deep-set() function is vulnerable to prototype pollution, allowing attackers to modify object prototypes and potentially achieve remote code execution or denial of service."
- redos: "The regex in no-case is vulnerable to ReDoS (Regular Expression Denial of Service), where specially crafted input causes exponential backtracking and excessive CPU consumption."

Typical fix patterns per vulnerability type (use these as recognition signals):

- redos: removing or replacing a catastrophic regex, adding input length limits before regex evaluation, replacing regex-based parsing with character-by-character or manual parsing, simplifying a regex to eliminate exponential backtracking
- prototype-pollution: wrapping property access in a safeGet() helper, filtering \texttt{__proto__} / \texttt{constructor} / \texttt{prototype} keys before assignment, using \texttt{Object.create(null)} for dictionaries, adding \texttt{hasOwnProperty} checks before merging
- command-injection: casting user-supplied input to integer (parseInt / Number / |0), switching from exec() to execFile() or spawnSync(), adding explicit type or format validation before a shell call
- code-injection: removing or sandboxing eval(), replacing dynamic property access that feeds eval, restricting the set of allowed expressions
- path-traversal: normalizing paths before use, rejecting inputs containing ../ or encoded traversal sequences, resolving the final path and asserting it is still inside the allowed base directory

Rules:

- If unsure of the exact API, provide the closest file path or symbol and be concise.
 - 'potential_vulnerability_type' must be exactly one of the five permitted values.
 - 'potential_vulnerable_API' must be the file path or module/function name that can be imported by using JavaScript import module.
 - Do NOT invent exploits, payloads, test cases, or external references.
 - If the commit does not clearly indicate a vulnerability but is plausibly security-related, choose the most plausible type and state a conservative description.
-